1. **To Install Linux VM, Configure GCC, Setup Git Repository, Create Project Structure, Understand Shell Architecture, Build Initial Makefile.**

Task -1: Create Project Structure as follows:

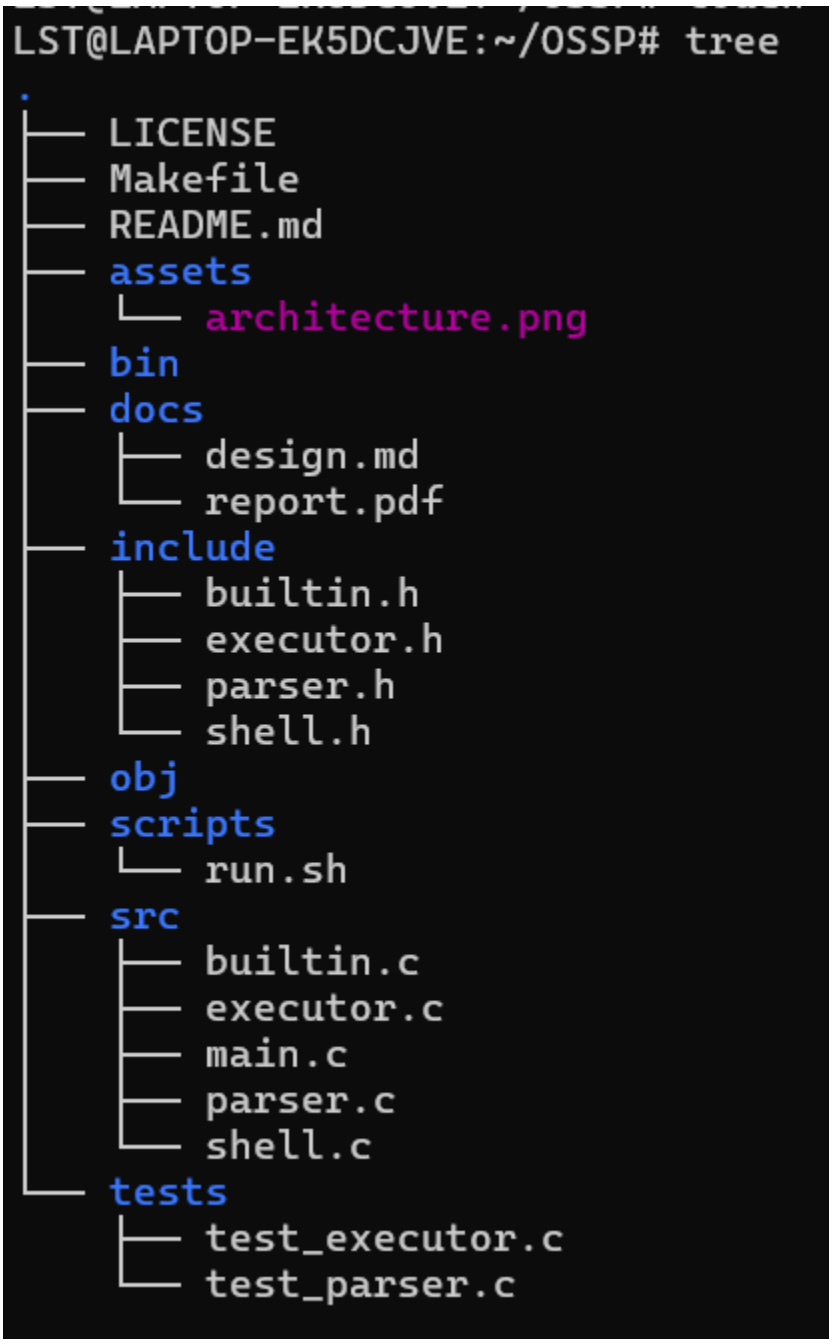
```
OSSP/
│
├── .git/                # Git repository (created automatically)
├── .gitignore           # Files/folders Git should ignore
├── README.md            # Project description and instructions
├── LICENSE              # Project license (optional)
├── Makefile             # Build instructions
│
├── src/                 # Source code
│   ├── main.c
│   ├── shell.c
│   ├── parser.c
│   ├── executor.c
│   └── builtin.c
│
├── include/             # Header files
│   ├── shell.h
│   ├── parser.h
│   ├── executor.h
│   └── builtin.h
│
├── obj/                 # Object files (.o) (generated)
│
├── bin/                 # Executable file (generated)
│   └── my_shell
│
├── docs/                # Documentation
│   ├── design.md
│   └── report.pdf
│
├── tests/               # Test programs
│   ├── test_parser.c
│   └── test_executor.c
│
├── scripts/             # Helper scripts
│   └── run.sh
│
└── assets/              # Images/screenshots
└── architecture.png
```

Note: For creating this structure use mkdir and touch commands.

```
LST@LAPTOP-EK5DCJVE:~/OSSP# mkdir src
LST@LAPTOP-EK5DCJVE:~/OSSP# mkdir include
LST@LAPTOP-EK5DCJVE:~/OSSP# mkdir obj
LST@LAPTOP-EK5DCJVE:~/OSSP# mkdir bin
LST@LAPTOP-EK5DCJVE:~/OSSP# mkdir docs
LST@LAPTOP-EK5DCJVE:~/OSSP# mkdir tests
LST@LAPTOP-EK5DCJVE:~/OSSP# mkdir scripts
LST@LAPTOP-EK5DCJVE:~/OSSP# mkdir assets
```

```
LST@LAPTOP-EK5DCJVE:~/OSSP# touch src/main.c
LST@LAPTOP-EK5DCJVE:~/OSSP# touch src/shell.c
LST@LAPTOP-EK5DCJVE:~/OSSP# touch src/parser.c
LST@LAPTOP-EK5DCJVE:~/OSSP# touch src/executor.c
LST@LAPTOP-EK5DCJVE:~/OSSP# touch src/builtin.c
LST@LAPTOP-EK5DCJVE:~/OSSP# touch include/shell.h
LST@LAPTOP-EK5DCJVE:~/OSSP# touch include/parser.h
LST@LAPTOP-EK5DCJVE:~/OSSP# touch include/executor.h
LST@LAPTOP-EK5DCJVE:~/OSSP# touch include/builtin.h
LST@LAPTOP-EK5DCJVE:~/OSSP# touch docs/design.md
LST@LAPTOP-EK5DCJVE:~/OSSP# touch docs/report.pdf
LST@LAPTOP-EK5DCJVE:~/OSSP# touch tests/test_parser.c
LST@LAPTOP-EK5DCJVE:~/OSSP# touch tests/test_executor.c
LST@LAPTOP-EK5DCJVE:~/OSSP# touch scripts/run.sh
LST@LAPTOP-EK5DCJVE:~/OSSP# touch assets/architecture.png
```

Output:



2. **To Analyze process abstraction, execute fork(), understand exec() family, analyze parent-child relationships, inspect process tree, practice system call tracing.**

   Task-1: Using a manual command understand the fork() and exec() system calls
   Task-2: Write a C program to create a new process and accepting the user commands using fork() and exec() system calls.

   Example:

```c
#include <stdio.h>
#include <unistd.h>
int main() {
    pid_t pid;
    pid = fork();   // Create a new process
    if (pid < 0) {
        printf("Fork failed!\n");
    }
    else if (pid == 0) {
        // Child Process
        printf("This is the Child Process.\n");
        printf("Child PID = %d\n", getpid());
    }
    else {
        // Parent Process
        printf("This is the Parent Process.\n");
        printf("Parent PID = %d\n", getpid());
        printf("Child PID = %d\n", pid);
    }
    return 0;
}
```

```
LST@LAPTOP-EK5DCJVE:~# gcc my_shell.c -o my_shell
LST@LAPTOP-EK5DCJVE:~# ./my_shell
This is the Parent Process.
Parent PID = 452
Child PID = 453
This is the Child Process.
Child PID = 453
LST@LAPTOP-EK5DCJVE:~#
```